# RSA

*Marcin Kocot*

Zadanie wykonano w języku Kotlin, przy użyciu build-toola Bazel.

---

# Konfiguracja RSA

Była przedstawiana przez klasę **RSA**, z wybraną długością składowej *modulus*. Długość czynnika *p* stanowiła jedną trzecią długości modulusa, a długość *q* dwie trzecie (tak, żeby suma się zgadzała, ale długości różniły się bardzo). Losowanie liczb odbyło się używając zaimplementowanego wcześniej generatora Blum-Blum-Shub.

```
class RSA(modulusBits: Int) {
  private val n: BigInteger  // modulus
  private val e: BigInteger  // public exponent
  private val d: BigInteger  // private exponent

  val publicKey: Key
  val privateKey: Key

  init {
    if (modulusBits < 512) {
      error("You must be stupid to use not even RSA-512!")
    }
    val pLen = modulusBits.floorDiv(3)
    val qLen = modulusBits - pLen
    val rGen: Random = BlumBlumShub()

    val p = BigInteger.probablePrime(pLen, rGen)
    val q = BigInteger.probablePrime(qLen, rGen)


    val f = lcm(p - BigInteger.ONE, q - BigInteger.ONE)

    // arbitrary - lower bit length ensures e < f
    val eLen = f.bitLength() - 3

    n = p * q
    e = BigInteger.probablePrime(eLen, rGen)
    d = e.modInverse(f)

    publicKey = Key(n, e)
    privateKey = Key(n, d)
  }

  private fun lcm(a: BigInteger, b: BigInteger): BigInteger =
    (a * b) / a.gcd(b)
}
```

Wiadomość była szyfrowana prostą funkcją (działającą w obie strony - szyfrowała gdy otrzymała klucz publiczny, odszyfrowywała gdy prywatny)

```
fun crypt(message: BigInteger): BigInteger =
    message.modPow(exponent, modulus)
```

---

# Uruchamianie

W każdym wystąpieniu `<duża liczba>`, prawdziwy output posiada dużą liczbę, ale zostały one tu usunięte celem poprawy czytelności sprawozdania. Testy wykonywano na tym samym tekście po Kantońsku. Testy wykonywane były dla `RSA(1024)`, ale wyższe długości modulusa też powinny zadziałać.

```
1. Key(modulus=<duża liczba>, exponent=<duża liczba>)
2. Key(modulus=<duża liczba>, exponent=<duża liczba>)
3. 香港人口稠密，擁有眾多島嶼，加上道路使用率之高，位居世界前列，為了應付龐大的人口交通流量，
香港興建了多座跨海大橋
4. e9a699e6b8afe4babae58fa3e7a8a0e5af86efbc8ce69381e69c89e79cbee...
5. 54fa70893ee76ea872c413675c7fc475f931bdb97ad6ce2869fbe61135a16...
6. e9a699e6b8afe4babae58fa3e7a8a0e5af86efbc8ce69381e69c89e79cbee...
7. 香港人口稠密，擁有眾多島嶼，加上道路使用率之高，位居世界前列，為了應付龐大的人口交通流量，
香港興建了多座跨海大橋
8. Do texts match? Yes!
```

1. Klucz publiczny
2. Klucz prywatny
3. Tekst otwarty
4. Binarna forma tekstu otwartego
5. Binarna forma tekstu zaszyfrowanego kluczem publicznym
6. Binarna forma tekstu odszyfrowanego kluczem prywatnym
7. Tekst odszyfrowany
8. Weryfikacja poprawności odszyfrowywania